Algoritmo CKY

Clase 22

IIC2223 / IIC2224

Prof. Cristian Riveros

Outline

Lema de bombeo para CFL (ant.)

Algoritmo CKY

Outline

Lema de bombeo para CFL (ant.)

Algoritmo CKY

Lema de bombeo para lenguajes libres de contexto

Sea $L \subseteq \Sigma^*$. Si L es libre de contexto entonces:

$(\mathrm{LB}^{\mathrm{CFL}})$ **existe** un $N > 0$ tal que
para toda palabra $z \in L$ con $|z| \geq N$
existe una descomposición $z = u\,v\,w\,x\,y$
con $vx \neq \epsilon$ y $|vwx| \leq N$ tal que
para todo $i \geq 0$, $u \cdot v^i \cdot w \cdot x^i \cdot y \in L$.

Sea $L \subseteq \Sigma^*$. Si:

$(\neg\,\mathrm{LB}^{\mathrm{CFL}})$ **para todo** $N > 0$ tal que
existe una palabra $z \in L$ con $|z| \geq N$
para toda descomposición $z = u\,v\,w\,x\,y$
con $vx \neq \epsilon$ y $|vwx| \leq N$ tal que
existe $i \geq 0$, $u \cdot v^i \cdot w \cdot x^i \cdot y \notin L$

entonces L **NO** es libre de contexto.

Jugando contra un demonio (versión CFL)





"L **NO es** CFL"

"L **es** CFL"

El escoge un $N > 0$

Uno escoge $z \in L$ **con** $|z| \geq N$

El escoge $u\,v\,w\,x\,y = z$ **con** $vx \neq \epsilon$ **y** $|vwx| \leq N$

Uno escoge $i \geq 0$

Uno gana si $u \cdot v^i \cdot w \cdot x^i \cdot y \notin L$

El gana si $u \cdot v^i \cdot w \cdot x^i \cdot y \in L$

Jugando contra un demonio ($a^n b^n c^n$)



"$a^n b^n c^n$ **NO es** CFL"



"$a^n b^n c^n$ **es** CFL"

Escojo $N > 0$

Yo escojo $a^N b^N c^N \in L$

Entonces escojo $uvwxy = a^N b^N c^N$ **con** $vx \neq \epsilon$ **y** $|vwx| \leq N$

Yo escojo $i = 2$

¿quién gana el juego?

Jugando contra un demonio $(a^n b^n c^n)$

Como $uvwxy = a^N b^N c^N$ con $vx \neq \epsilon$ y $|vwx| \leq N$, entonces:

$$vwx \in \mathcal{L}(a^* b^*) \quad \text{o} \quad vwx \in \mathcal{L}(b^* c^*)$$

¿por qué?

- Si $vwx \in \mathcal{L}(a^* b^*)$, entonces:
  - $\#_a(uv^2wx^2y) + \#_b(uv^2wx^2y) > 2N$
  - $\#_c(uv^2wx^2y) = N$

  por lo tanto, $z' \notin L$.

- Si $vwx \in \mathcal{L}(b^* c^*)$, entonces:
  - $\#_b(uv^2wx^2y) + \#_c(uv^2wx^2y) > 2N$
  - $\#_a(uv^2wx^2y) = N$

  por lo tanto $z' \notin L$.

En ambos casos, $uv^2wx^2y \notin L$

Jugando contra un demonio

$(\neg\, \mathrm{LB}^{\mathrm{CFL}})$ **para todo** $N > 0$ tal que
existe una palabra $z \in L$ con $|z| \geq N$
para toda descomposición $z = u\,v\,w\,x\,y$
con $vx \neq \epsilon$ y $|vwx| \leq N$ tal que
existe $i \geq 0$, $u \cdot v^i \cdot w \cdot x^i \cdot y \notin L$.

entonces L **NO** es libre de contexto.

Lema de bombeo (version juego)

"Dado un lenguaje $L \subseteq \Sigma^*$, si **UNO** *tiene una estrategia ganadora en el juego $(\neg\, \mathrm{LB}^{\mathrm{CFL}})$ para toda estrategia posible del* **demonio**, *entonces L* **NO** *es libre de contexto."*

Consecuencias: unión, intersección y complemento

Proposición

Para todo lenguajes libres de contexto L_1 y L_2,
$L_1 \cup L_2$ es un lenguaje libre de contexto.

Existen lenguajes libres de contexto L, L_1 y L_2:

- $L_1 \cap L_2$ **NO** es un lenguaje libre de contexto.
- L^c **NO** es un lenguaje libre de contexto.

Demostración

$$
\begin{aligned}
L_1 &= \left\{ a^n b^n c^m \mid n \geq 0, m \geq 0 \right\} \\
L_2 &= \left\{ a^m b^n c^n \mid n \geq 0, m \geq 0 \right\}
\end{aligned}
$$

¿son L_1 y L_2 lenguajes libres de contexto? ¿y $L_1 \cap L_2$?

Ejercicio: demuestre el caso de L^c.

Outline

Lema de bombeo para CFL (ant.)

Algoritmo CKY

¿cómo verificar si $w \in \mathcal{L}(\mathcal{G})$?

Dado un lenguaje libre de contexto L y una palabra w:

¿cómo verificamos si $w \in L$?

¿cómo determinar si la palabra $bbaba \in \mathcal{L}(\mathcal{G})$?

$$
\begin{array}{lcl}
\mathsf{S} & \rightarrow & \mathsf{XY} \mid \mathsf{YZ} \\
\mathsf{X} & \rightarrow & \mathsf{YY} \mid \mathsf{a} \\
\mathsf{Y} & \rightarrow & \mathsf{YX} \mid \mathsf{b} \\
\mathsf{Z} & \rightarrow & \mathsf{XZ} \mid \mathsf{XX} \mid \mathsf{a}
\end{array}
$$

¿cómo verificar si $w \in \mathcal{L}(\mathcal{G})$?

Dado un lenguaje libre de contexto L y una palabra w:

¿cómo verificamos si $w \in L$?

Posible estrategia

- Convertimos $\mathcal{G}$ en formal normal de Chomsky.
- Probamos todas las derivaciones de altura a lo mas $|w| + 1$.
- Si encontramos una derivación retornamos `TRUE`.

¿cuántas derivaciones debemos probar?

Algoritmo CKY

- Inventado por:

  John **C**ocke
  Tadao **K**asami
  Daniel **Y**ounger

- Algoritmo **cúbico** en $|w|$ y **lineal** en $|\mathcal{G}|$:

  Tiempo: $\mathcal{O}(|w|^3 \cdot |\mathcal{G}|)$

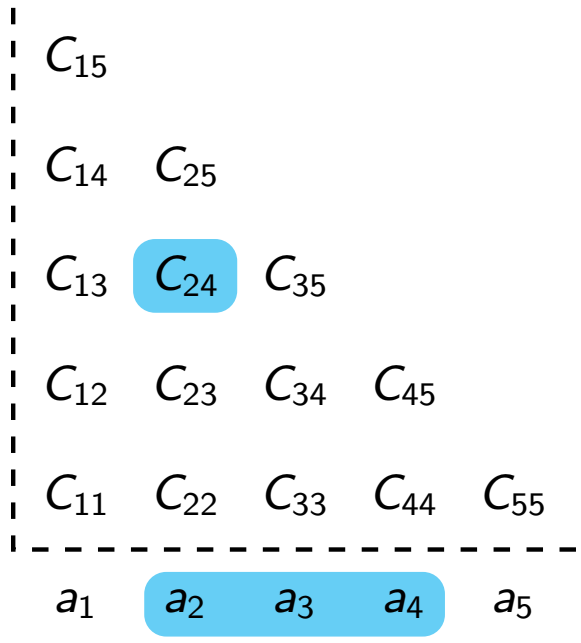
- Un ejemplo del uso de **programación dinámica**.

Por simplicidad asumiremos que
las gramáticas esta en **Forma Normal de Chomsky** (CNF)

. . . CKY se puede adaptar para producciones mayores que 2

Tabla del algoritmo CKY

Para una palabra $w = a_1 a_2 \dots a_n$ y una gramática $\mathcal{G} = (V, \Sigma, P, S)$ construimos la **tabla CKY**:



- Para todo $1 \leq i \leq j \leq n$ se define:

$$C_{ij} = \{ X \in V \mid X \overset{\star}{\underset{\mathcal{G}}{\Rightarrow}} a_i \dots a_j \}$$

Tabla del algoritmo CKY

Ejemplo

Considere la palabra *bbaba* y la gramática:

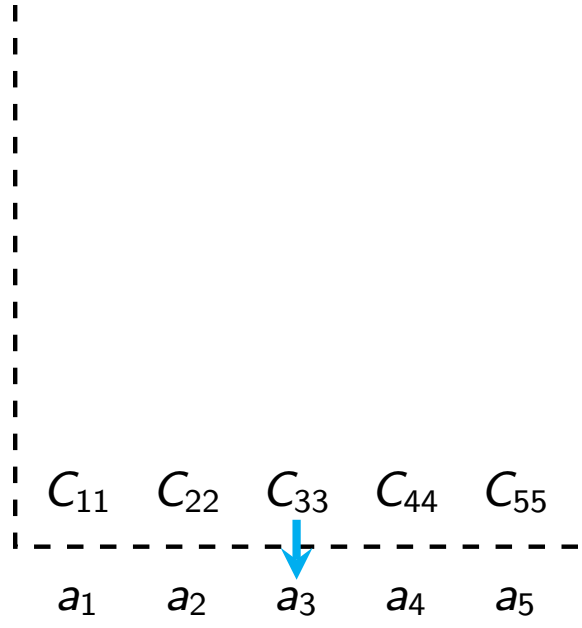
$$
\begin{array}{lcl}
\mathrm{S} & \to & \mathrm{XY} \mid \mathrm{YZ} \\
\mathrm{X} & \to & \mathrm{YY} \mid \mathrm{a} \\
\mathrm{Y} & \to & \mathrm{YX} \mid \mathrm{b} \\
\mathrm{Z} & \to & \mathrm{XZ} \mid \mathrm{XX} \mid \mathrm{a}
\end{array}
$$

$\{S, Y\}$				
$\{S, Y\}$	$\{X, Z\}$			
$\{X, Z\}$	$\{X\}$	$\{S\}$		
$\{X\}$	$\{S, Y\}$	$\{S\}$	$\{S, Y\}$	
$\{Y\}$	$\{Y\}$	$\{X, Z\}$	$\{Y\}$	$\{X, Z\}$
b	b	a	b	a

¿cómo **verificamos** si $w \in \mathcal{L}(\mathcal{G})$ usando la tabla CKY?
¿cómo **construimos** la tabla CKY?

Algoritmo CKY: construcción de la tabla CKY

Paso 0 (inicial)



Para cada i, construimos el conjunto $C_{ii} \subseteq V$ tal que:

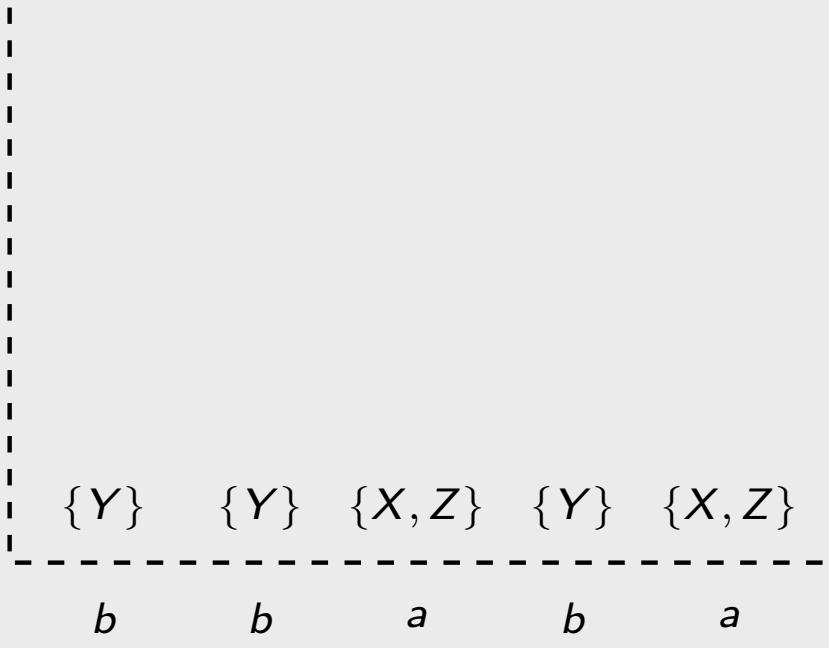
$$C_{ii} \quad = \quad \left\{ X \in V \mid X \to a_i \in P \right\}$$

Algoritmo CKY: construcción de la tabla CKY

Ejemplo (Paso 0)

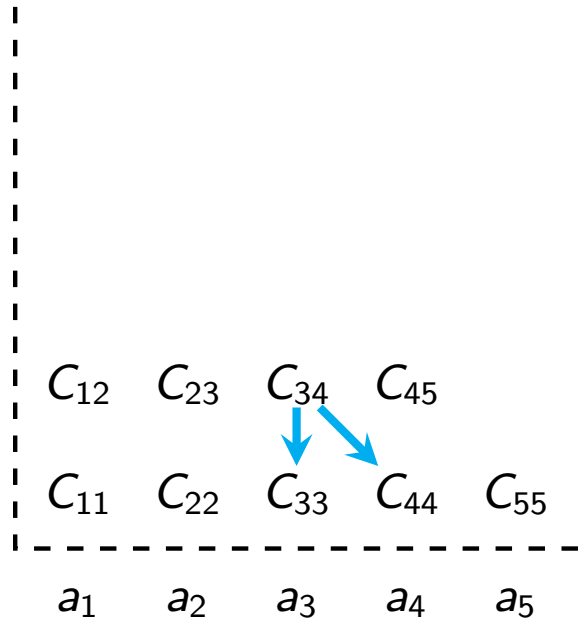
Considere la palabra *bbaba* y la gramática:

$$
\begin{array}{lcl}
\mathsf{S} & \to & \mathsf{XY} \mid \mathsf{YZ} \\
\mathsf{X} & \to & \mathsf{YY} \mid \mathsf{a} \\
\mathsf{Y} & \to & \mathsf{YX} \mid \mathsf{b} \\
\mathsf{Z} & \to & \mathsf{XZ} \mid \mathsf{XX} \mid \mathsf{a}
\end{array}
$$



Algoritmo CKY: construcción de la tabla CKY

Paso 1



Para cada i, construimos el conjunto $C_{i\,i+1} \subseteq V$ tal que:

$$C_{i\,i+1} = \left\{ X \in V \mid X \to YZ \in P \text{ para algún } Y \in C_{ii} \wedge Z \in C_{i+1\,i+1} \right\}$$

Algoritmo CKY: construcción de la tabla CKY

Ejemplo (Paso 1)

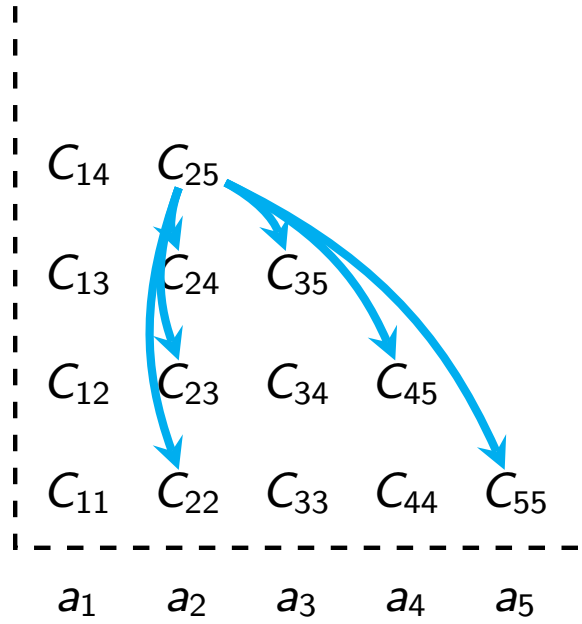
Considere la palabra *bbaba* y la gramática:

$$
\begin{array}{lcl}
\mathsf{S} & \to & \mathsf{XY} \mid \mathsf{YZ} \\
\mathsf{X} & \to & \mathsf{YY} \mid \mathsf{a} \\
\mathsf{Y} & \to & \mathsf{YX} \mid \mathsf{b} \\
\mathsf{Z} & \to & \mathsf{XZ} \mid \mathsf{XX} \mid \mathsf{a}
\end{array}
$$

$\{X\}$	$\{S, Y\}$	$\{S\}$	$\{S, Y\}$	
$\{Y\}$	$\{Y\}$	$\{X, Z\}$	$\{Y\}$	$\{X, Z\}$
b	b	a	b	a

Algoritmo CKY: construcción de la tabla CKY

Paso k $(k > 0)$)

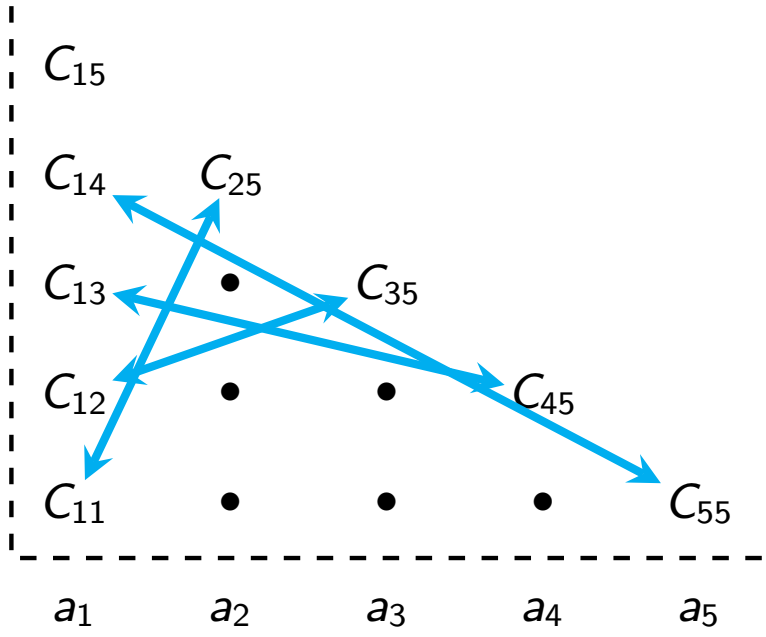


Para cada i, construimos el conjunto $C_{i\,i+k} \subseteq V$ tal que:

$$C_{i\,i+k} = \left\{ X \in V \;\middle|\; \exists j \in [i, i+k). \quad X \to YZ \in P \text{ para algún } Y \in C_{ij} \wedge Z \in C_{j+1\,i+k} \right\}$$

Algoritmo CKY: construcción de la tabla CKY

Paso k $(k > 0)$)



Para cada i, construimos el conjunto $C_{i\,i+k} \subseteq V$ tal que:

$$C_{i\,i+k} = \left\{ X \in V \;\middle|\; \exists j \in [i, i+k). \quad X \to YZ \in P \text{ para algún } Y \in C_{ij} \wedge Z \in C_{j+1\,i+k} \right\}$$

Algoritmo CKY: construcción de la tabla CKY

Ejemplo (Paso k)

Considere la palabra *bbaba* y la gramática:

$$
\begin{array}{lcl}
\mathsf{S} & \to & \mathsf{XY} \mid \mathsf{YZ} \\
\mathsf{X} & \to & \mathsf{YY} \mid \mathsf{a} \\
\mathsf{Y} & \to & \mathsf{YX} \mid \mathsf{b} \\
\mathsf{Z} & \to & \mathsf{XZ} \mid \mathsf{XX} \mid \mathsf{a}
\end{array}
$$

$\{S, Y\}$				
$\{S, Y\}$	$\{X, Z\}$			
$\{X, Z\}$	$\{X\}$	$\{S\}$		
$\{X\}$	$\{S, Y\}$	$\{S\}$	$\{S, Y\}$	
$\{Y\}$	$\{Y\}$	$\{X, Z\}$	$\{Y\}$	$\{X, Z\}$
b	b	a	b	a

Algoritmo CKY: construcción de la tabla CKY

Otro ejemplo

Considere la palabra *abba* y la gramática:

S	$\rightarrow$	XY | YX | SS | XZ | YW
X	$\rightarrow$	a
Y	$\rightarrow$	b
Z	$\rightarrow$	SY
W	$\rightarrow$	SX

$\{S\}$			
$\{Z\}$	$\varnothing$		
$\{S\}$	$\varnothing$	$\{S\}$	
$\{X\}$	$\{Y\}$	$\{Y\}$	$\{X\}$
a	*b*	*b*	*a*

Algoritmo CKY

input : Una gramática $\mathcal{G} = (V, \Sigma, P, S)$ y una palabra $w = a_1 a_2 \dots a_n$

output: `TRUE` ssi $w = a_1 a_2 \dots a_n \in \mathcal{L}(\mathcal{G})$

Function `AlgoritmoCKY` $(\mathcal{G}, w)$

- **for** $i \leftarrow 1$ **to** n **do**
  - **let** $C_{ii} \leftarrow \varnothing$
  - **for** $X \to c \in P$ **do**
    - **if** $c = a_i$ **then let** $C_{ii} = C_{ii} \cup \{X\}$
- **for** $k \leftarrow 1$ **to** $n - 1$ **do**
  - **for** $i \leftarrow 1$ **to** $n - k$ **do**
    - **let** $C_{i\,i+k} \leftarrow \varnothing$
    - **for** $j \leftarrow i$ **to** $i + k - 1$ **do**
      - **for** $X \to YZ \in P$ **do**
        - **if** $Y \in C_{i\,j} \wedge Z \in C_{j+1\,i+k}$ **then let**
          - $C_{i\,i+k} \leftarrow C_{i\,i+k} \cup \{X\}$
- **return check** $S \in C_{1n}$

Análisis algoritmo CKY

Correctitud algoritmo CKY

Para toda gramática $\mathcal{G}$ y para toda palabra $w \in \Sigma^*$ se tiene que:

$$\texttt{AlgoritmoCKY}(\mathcal{G}, w) = \texttt{TRUE} \quad \text{ssi} \quad w \in \mathcal{L}(\mathcal{G})$$

Demostración (ejercicio)

Si el input es de tamaño $|w|$ y la gramática es de tamaño $|\mathcal{G}|$, entonces:

Tiempo Algoritmo CKY: $\mathcal{O}(|w|^3 \cdot |\mathcal{G}|)$

¿es posible verificar si $w \in \mathcal{L}(\mathcal{G})$ en **tiempo lineal** en $|w|$?

Cierre de clase

En esta clase vimos:

- Evaluación de gramáticas libres de contexto
- Algoritmo CKY con tiempo $\mathcal{O}(|w|^3 \cdot |\mathcal{G}|)$

Próxima clase: Hacia algoritmos eficientes de evaluación de gramáticas